

Technical Design — Multi-Channel Inventory Hub

Audience: web developers, backend engineers, DBAs, and open-source contributors.

1. Goals & Non-Goals

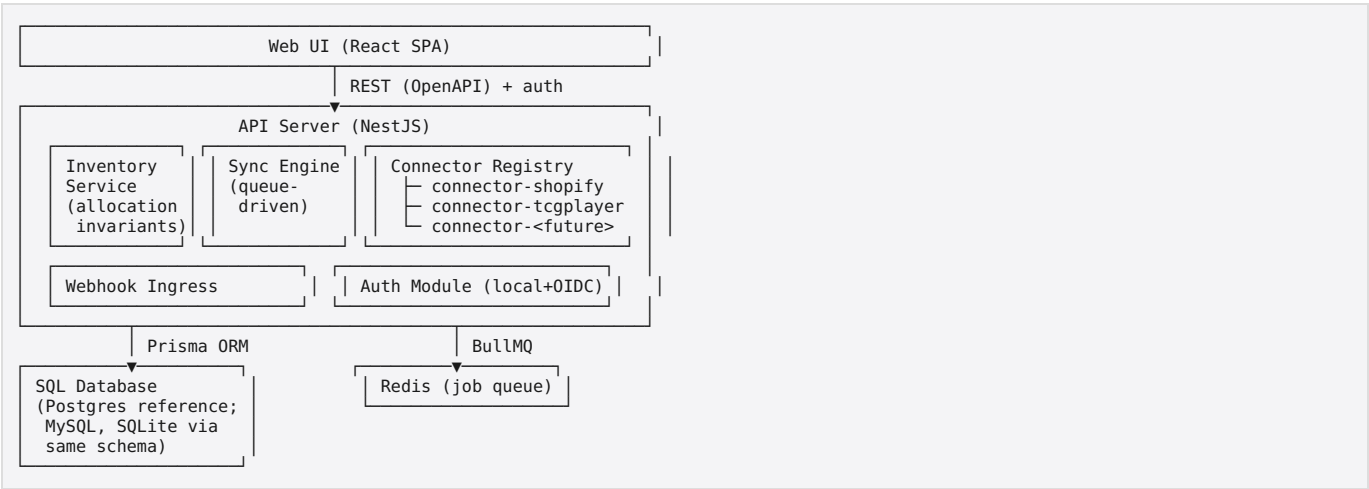
Goals

- A self-hostable, open-source inventory system where an internal ledger is the single source of truth and sales channels (Shopify, TCGPlayer, future marketplaces) are pluggable "connectors."
- Independent per-channel quantity allocation and per-channel pricing for each SKU.
- Database-agnostic persistence: Postgres as the reference database, with MySQL/MariaDB and SQLite supported through the ORM layer.
- A web UI for browsing, searching, and editing inventory, with pluggable authentication.
- Docker-first deployment; cloud-friendly but never cloud-required.

Non-Goals (v1)

- POS, shipping/fulfillment, automated repricing, multi-tenant SaaS hosting.
- Supporting non-relational databases. The domain (allocations, mappings, ledgers) is strongly relational.

2. Architecture Overview



Key principles:

- **Webhook ingress does no work inline.** It validates signatures, persists a raw event, enqueues a job, and returns 200 immediately. All processing happens in queue workers.
- **Connectors are dumb pipes; the core owns quantity math.** A connector never decides quantities — it translates between the core's canonical operations and a platform's API.
- **Every external mutation is logged** to an append-only `sync_events` table before and after execution.

3. Tech Stack & Rationale

Layer	Choice	Rationale
Language	TypeScript (Node 20+ LTS)	One language across API, UI, and connector SDK; largest OSS contributor pool; Shopify's own tooling is TS-first.

API framework	NestJS (Fastify adapter)	DI + module system maps cleanly to a plugin/connector architecture; well-documented for contributors.
ORM	Prisma	Single schema targeting Postgres, MySQL/MariaDB, SQLite, SQL Server; type-safe client; mature migration tooling.
Job queue	BullMQ (Redis)	Battle-tested delayed jobs, retries with backoff, rate limiting — all essential for marketplace APIs.
Web UI	React + Vite, TanStack Query/Table/Router	SPA served as static assets by the API container; no SSR complexity for a self-hosted admin tool.
API contract	REST + OpenAPI (generated from NestJS decorators)	Enables generated typed clients; friendlier to third-party integrations than tRPC.
Monorepo	pnpm workspaces	apps/api, apps/web, packages/connector-sdk, packages/connector-shopify, packages/connector-tcgplayer, packages/db.

Database-agnosticism rules for contributors:

- No raw SQL in application code; Prisma queries only. Where raw SQL is unavoidable (reporting), gate it behind a per-dialect adapter.
- No dialect-specific column types in the schema (no Postgres arrays/JSONB-specific operators in query paths; JSON columns allowed since all target DBs support them, but never queried by JSON-path in core logic).
- CI runs the full test suite against Postgres, MySQL, and SQLite matrices.

4. Data Model

Core entities (Prisma-style, abbreviated):

```

model CatalogItem {
  id          String @id @default(uuid())
  name        String
  game        String? // e.g. "Magic", "Pokemon"; null for non-TCG goods
  setName     String?
  externalRefs CatalogExternalRef[] // canonical IDs on each platform
  skus        Sku[]
}

model CatalogExternalRef { // e.g. tcgplayer productId – reuse canonical IDs
  id          String @id @default(uuid())
  catalogItemId String
  source       String // "tcgplayer", "scryfall", ...
  externalId   String
  @@unique([source, externalId])
}

model Sku {
  id          String @id @default(uuid())
  catalogItemId String
  condition   String // NM | LP | MP | HP | DMG | SEALED | NA
  printing    String? // foil, 1st edition, etc.
  language    String @default("EN")
  inventory   InventoryItem?
}

model InventoryItem {
  id          String @id @default(uuid())
  skuId       String @unique
  quantityOnHand Int // physical truth
  reserveQuantity Int @default(0) // held back from all pooled listings
  costBasis   Int? // cents; optional
  allocations ChannelAllocation[]
  // INVARIANT: quantityOnHand >= SUM(fixed allocations) + reserveQuantity >= 0
  // (see "Allocation Modes" below)
}

model ChannelAllocation { // Per-channel listing: quantity mode + price
  id          String @id @default(uuid())
  inventoryItemId String
  channelId   String // FK -> ChannelInstance (a configured connector)
  mode        String // "fixed" | "pooled" (see Allocation Modes below)
  quantityAllocated Int? // fixed mode only: exclusive partition size
  maxQuantity Int? // pooled mode only: optional cap; null = mirror all
  listedQuantity Int // cached last-pushed qty (computed in pooled mode)
  price       Int // cents, channel-specific
  currency    String @default("USD")
  externalListingId String? // shopify variant id / tcgplayer sku listing id
  status      String // draft | listed | error | delisted
}

```

```

    @unique([inventoryItemId, channelInstanceId])
  }

  model ChannelInstance {
    id          String @id @default(uuid())
    connectorKey String // "shopify", "tcgplayer" — resolves a plugin
    displayName String
    config      Json      // non-secret connector config
    credentialRef String    // pointer into encrypted credential store
    enabled     Boolean @default(true)
  }

  model SyncEvent {
    // Append-only audit/debug log
    id          String @id @default(uuid())
    ts          DateTime @default(now())
    direction   String // inbound | outbound | reconcile
    channelInstanceId String?
    entityType  String // allocation | order | listing
    entityId    String?
    payload     Json    // raw request/response or webhook body
    outcome     String  // ok | error | conflict
    detail      String?
  }

```

Plus standard `User`, `Role`, `Session`/`ApiKey` tables for auth (see §8).

Allocation Modes

Each channel allocation operates in one of two modes, and modes can be mixed on the same SKU:

- **fixed (partitioned)**: the channel owns an exclusive slice of stock. A sale on this channel decrements only its own partition (plus on-hand). Use when stock must be strictly separated per channel.
- **pooled (mirrored)**: the channel lists from the shared remainder. Its listed quantity is **computed**, never stored as truth:

```

pool          = quantityOnHand - Σ(fixed partitions) - reserveQuantity
listed(chan)  = min(chan.maxQuantity ?? ∞, pool)

```

With no cap, the channel mirrors the full pool (e.g., 10 on hand → 10 listed on Shopify *and* 10 on TCGPlayer). With a cap, the channel never exposes more than the cap (10 on hand, Shopify capped at 5, TCGPlayer uncapped → 5 / 10) — the cap effectively insulates stock from that channel without a separate reservation system. A capped channel's listing only shrinks once the pool falls below its cap.

- **reserveQuantity** (optional, per `InventoryItem`, default 0): stock subtracted from the pool before any pooled math — "never list the last N anywhere."

Sale semantics by mode: a sale on a *fixed* channel decrements that partition + on-hand, then pushes updates only to other channels sharing the SKU via the pool. A sale on a *pooled* channel decrements on-hand, then recomputes and pushes `listed` to **every** pooled channel on that SKU. Pooled mode intentionally double-lists the same physical units across channels, so the oversell race window is inherent to the mode (not a defect); the pessimistic conflict policy and oversell alerting in §6 are the designed handling for it.

Invariant enforcement: the core invariant becomes $\text{quantityOnHand} \geq \Sigma(\text{fixed quantityAllocated}) + \text{reserveQuantity} \geq 0$; pooled `listedQuantity` values are derived and therefore cannot violate it. All quantity mutations go through a single `InventoryService` using DB transactions with row-level locking (`SELECT ... FOR UPDATE` semantics via Prisma interactive transactions) so concurrent sale events on different channels can't both claim the last unit. A CHECK constraint backs this up on databases that support it; the service layer is the portable guarantee.

5. Connector SDK

Connectors are npm packages implementing a published interface from `@hub/connector-sdk`. The core discovers them via a registry (bundled connectors registered at startup; future: dynamic loading).

```

interface Connector {
  key: string; // "shopify"
  displayName: string;
  configSchema: JSONSchema; // drives auto-generated settings UI
  capabilities: Capability[]; // declared, not assumed

  // Catalog (optional capability)
  searchCatalog?(q: CatalogQuery): Promise<CatalogResult[]>;

```

```

// Outbound — core calls these; connector translates to platform API
pushListing(ctx: Ctx, req: PushListingRequest): Promise<PushListingResult>;
updateQuantity(ctx: Ctx, req: UpdateQtyRequest): Promise<void>;
updatePrice(ctx: Ctx, req: UpdatePriceRequest): Promise<void>;
delist(ctx: Ctx, req: DelistRequest): Promise<void>;

// Inbound — connector normalizes platform events into core events
verifyWebhook?(headers: Headers, rawBody: Buffer): boolean;
parseWebhook?(rawBody: Buffer): NormalizedEvent[]; // -> SaleEvent, etc.
pollChanges?(ctx: Ctx, since: Date): Promise<NormalizedEvent[]>; // backstop

// Reconciliation
fetchLiveState(ctx: Ctx, listingIds: string[]): Promise<LiveListingState[]>;
}

type Capability =
  | "catalog.search" | "listing.push" | "listing.price" | "listing.quantity"
  | "orders.webhook" | "orders.poll" | "reconcile";

```

Design notes:

- **Capabilities are declared** so the core UI/engine degrades gracefully — e.g., a connector without `orders.webhook` is automatically scheduled for `pollChanges`.
- `configSchema` is **JSON Schema**, so the settings UI for any connector (including community ones) is generated, not hand-built.
- **Credentials never live in `config`**. The core provides an encrypted credential store (AES-GCM with a server master key from env/KMS); connectors receive decrypted secrets only inside `ctx` at call time.
- **Rate limiting and retries live in the core queue layer**, configured per connector (declared limits), so individual connectors stay simple.

Initial connectors:

- `connector-shopify`: Admin GraphQL API; webhooks for `orders/create` and `inventory_levels/update`; HMAC webhook verification.
- `connector-tcgplayer`: OAuth 2.0 authorization-code flow (requires approved API application + Pro seller account); catalog/pricing endpoints usable pre-approval for `catalog.search`; inventory push + order polling post-approval. Polling is the primary inbound mechanism unless/until webhook support proves reliable.

6. Sync Engine

All sync work flows through named BullMQ queues with per-channel-instance rate limiters.

Outbound flow (user edits allocation → channel):

1. `InventoryService` mutates allocation in a transaction and writes a `SyncEvent(outbound, pending)`.
2. Enqueues `push:{channelInstanceId}` job.
3. Worker resolves connector, calls `updateQuantity/pushListing`, records outcome. Retries with exponential backoff; terminal failures set `allocation.status = error` and raise an alert.

Inbound flow (sale on a channel):

1. Webhook ingress verifies + persists raw event, enqueues `inbound` job (or `pollChanges` produces the same `NormalizedEvents` on schedule).
2. Worker maps external listing ID → allocation, then in one transaction: decrement `quantityOnHand`, and if the selling allocation is `fixed`, also decrement its `quantityAllocated`.
3. Recompute listed quantities for **all** allocations on the SKU per their modes (§4 Allocation Modes) and enqueue outbound quantity pushes for every channel whose listed value changed — in pooled mode this fans out to every mirrored channel on each sale.
4. If the decrement would go negative (race: sold elsewhere first): clamp to zero, mark `SyncEvent(conflict)`, raise an **oversell alert** for human resolution. We do not attempt automated cancellation.

Conflict policy: last-write-wins on price; quantity conflicts always resolve pessimistically (lower quantity wins) and alert.

Reconciliation (scheduled, default nightly + on-demand):

- For each enabled channel instance, `fetchLiveState` for all listed allocations.
- Diff against ledger. Any mismatch → `SyncEvent(reconcile, conflict)` + alert with both values. Auto-correction is opt-in per

channel and only in the "push our ledger to the channel" direction — the ledger is never silently rewritten from a channel.

7. Web UI

Views for v1:

- **Inventory browser**: server-side paginated/sorted/filtered table (SKU, condition, on-hand, unallocated, per-channel qty/price/status). Saved filters.
- **Item detail**: allocation editor (adjust per-channel quantity + price with live invariant validation), sync history for that SKU.
- **Intake flow**: search catalog (TCGPlayer catalog connector) → select SKU/condition → set quantity → lands as *unallocated* stock.
- **Channels**: configure channel instances via generated forms from `configSchema`; OAuth connect flows; health status.
- **Sync activity**: filterable `SyncEvent` stream; conflict/alert inbox.
- **Query console** (admin-only, opt-in via env flag): read-only SQL against a restricted role/connection. Writes from the UI go exclusively through the API's validated endpoints — never raw SQL — to protect the allocation invariant. This satisfies "run queries via the web" without letting ad-hoc SQL corrupt the ledger.

8. Authentication & Authorization

Pluggable auth behind a single `AuthProvider` interface:

- **v1: Local provider** — username/password (argon2id), session cookies (SameSite=Lax, httpOnly), CSRF protection. First-run setup creates the initial admin.
- **v1.x: Generic OIDC provider** — works with Keycloak, Auth0, Google, Entra, etc. via standard discovery. Chosen over per-vendor OAuth so self-hosters plug in anything.
- **API keys** for headless/automation access, scoped to roles.

RBAC roles: `admin` (everything, incl. channels/credentials/query console), `editor` (inventory + allocations), `viewer` (read-only). Enforced via NestJS guards on every route; the UI only reflects, never enforces.

9. Deployment

Docker-first:

- `docker-compose.yml` ships in-repo: `app` (API + static UI), `postgres`, `redis`. One command to a running dashboard.
- Single multi-arch image (amd64/arm64) published on release via GitHub Actions.
- All config via environment variables (12-factor); `.env.example` documented. `DATABASE_URL` selects the dialect — Postgres/MySQL/SQLite supported by the same image.

Cloud notes: stateless app container scales horizontally; Redis and the DB are the only stateful services, both replaceable with managed equivalents (RDS/Cloud SQL, Elasticache/Upstash). BullMQ workers can run in the app process (default) or as a separate worker deployment via env flag.

10. Repository Layout & OSS Hygiene

```
/apps/api      NestJS server (serves /apps/web build)
/apps/web     React SPA
/packages/db   Prisma schema + migrations + seed
/packages/connector-sdk
/packages/connector-shopify
/packages/connector-tcgplayer
/docs         This document, connector-authoring guide, ADRs
```

- License: MIT (or AGPL if preventing closed-source SaaS forks matters — decide before first public release).
- CI: lint, typecheck, unit + integration tests against the 3-DB matrix, connector contract tests (every connector must pass a shared test suite against a mock platform).

- `CONNECTOR_GUIDE.md`: the contract tests double as the spec for community connector authors (eBay, Square, Cardmarket...).

11. Phased Delivery Plan

Phase	Scope	Notes
0	Monorepo scaffold, CI, Docker compose, auth (local), DB schema + migrations	Submit TCGPlayer Pro + API application on day 1 — longest external lead time in the project.
1	Inventory CRUD + allocation invariants + inventory browser/detail UI	Usable standalone inventory tool.
2	Connector SDK + TCGPlayer catalog search (pre-approval endpoints) + intake flow	
3	Shopify connector: outbound push, then inbound webhooks; sync activity UI	Proves the full sync pattern end-to-end.
4	TCGPlayer connector: OAuth, outbound push, inbound polling	Gated on API approval; reuses Phase 3 patterns.
5	Reconciliation + alerting, query console, OIDC provider, docs polish, public release	

12. Risks & Mitigations (Technical)

- **TCGPlayer API approval timeline is external.** Mitigated by phase ordering; connector contract tests run against a mock TCGPlayer implementation so the connector can be built and verified pre-approval.
- **Race conditions on shared-SKU sales.** Mitigated by transactional decrements with row locking, pessimistic quantity conflict policy, and oversell alerting. Accepted residual risk: sub-minute windows between a sale and the cross-channel push.
- **Prisma dialect drift.** Mitigated by the 3-DB CI matrix and the "no raw SQL in core" rule.
- **Community connector quality.** Mitigated by capability declarations, shared contract tests, and credential isolation in the core.